

Application-Centric AI Evals for Engineers and Technical PMs

Shreya Shankar and Hamel Husain

Summer 2025

Evaluating the complex, often subjective outputs of Large Language Models (LLMs) presents unique challenges distinct from traditional software testing or ML validation. We present a framework for application-centric LLM evaluation: the *Analyze-Measure-Improve* lifecycle.

Contents

1	Introduction	4
1.1	What is Evaluation?	5
1.2	The Three Gulfs of LLM Pipeline Development	5
1.3	Why LLM Pipeline Evaluation is Challenging	7
1.4	The LLM Evaluation Lifecycle: Bridging the Gulfs with Evaluation	8
1.5	Summary	9
2	LLMs, Prompts, and Evaluation Basics	11
2.1	Strengths and Weaknesses of LLMs	11
2.2	Prompting Fundamentals	12
2.3	Defining “Good”: Types of Evaluation Metrics	15
2.4	Foundation Models vs. Application-Centric Evals	16
2.5	Eliciting Labels for Metric Computation	17
2.6	Summary	18
2.7	Glossary of Terms	19
2.8	Exercises	19
3	Error Analysis	22
3.1	Create a Starting Dataset	22
3.2	Open Coding: Read and Label Traces	26
3.3	Axial Coding: Structuring and Merging Failure Modes	29
3.4	Labeling Traces after Structuring Failure Modes	30
3.5	Iteration and Refining the Failure Taxonomy	31
3.6	Common Pitfalls	32
3.7	Summary	33
3.8	Exercises	34
4	Collaborative Evaluation Practices	40
4.1	“Benevolent Dictators” Are Sometimes Preferable	40
4.2	A Collaborative Annotation Workflow	41
4.3	Measuring Inter-Annotator Agreement (IAA)	42

4.4	Facilitating Alignment Sessions and Resolving Disagreements	44
4.5	Connecting Collaborative Labels to Automated Evaluators	44
4.6	Common Pitfalls in Collaborative Evaluation	45
4.7	Summary	46
4.8	Exercises	46
5	Implementing Automated Evaluators	54
5.1	Defining the Right Metrics (What to Measure)	54
5.2	Implementing Metrics (How to Measure)	57
5.3	Writing LLM-as-Judge Prompts	58
5.4	Data Splits for Designing and Validating LLM-as-Judge	61
5.5	Iterative Prompt Refinement for the LLM-as-Judge	61
5.6	Estimating True Success Rates with Imperfect Judges	63
5.7	Python Code for Estimating Success Rates	65
5.8	Optional: Group-wise Metrics for Evaluating Multiple Outputs	68
5.9	Common Pitfalls	70
5.10	Summary	71
5.11	Exercises	71
6	Evaluating Multi-Turn Conversations	79
6.1	Evaluating at Different Levels	79
6.2	Practical Strategies for Multi-Turn Evaluation	79
6.3	Automated Evaluation of Multi-Turn Traces	82
6.4	Addressing Common Pitfalls	82
6.5	Summary	82
7	Evaluating Retrieval-Augmented Generation (RAG)	84
7.1	Overview	84
7.2	Synthetically Generating Query-Answer Pairs	85
7.3	Metrics for Retrieval Quality	88
7.4	Evaluating and Optimizing Chunking Strategies	90
7.5	Evaluating Generation Quality	91
7.6	Common Pitfalls	92
7.7	Summary	93
7.8	Exercises	93
8	Specific Architectures and Data Modalities	104
8.1	Tool Calling	104
8.2	Agentic Systems	106
8.3	Debugging Multi-Step Pipelines	107
8.4	Evaluating Specific Input Data Modalities	110
8.5	Common Pitfalls	113
8.6	Summary	114
8.7	Exercises	114

9	Continuous Integration and Deployment	122
9.1	CI: Building a Safety Net Against Regressions	122
9.2	CD & Online Monitoring: Tracking Real-World Performance	125
9.3	The Continuous Improvement Flywheel	127
9.4	Practical Considerations and Common Pitfalls for Production LLM Evaluation	130
9.5	Summary	131
10	Interfaces for Continuous Human Review and Error Analysis	133
10.1	The Case for Custom Review Interfaces	133
10.2	Principles of Effective Review Interfaces	134
10.3	Case Study: EvalGen Interface and Insight	136
10.4	Selecting Traces from the “Firehose” for Human Review . .	137
10.5	Navigating Trace Groups and Discovering Patterns	138
10.6	Integrating Human Review into the Bigger Engineering Workflow	139
10.7	Example Walkthrough: Reviewing Real Estate Assistant Emails	140
10.8	Case Study: DocWrangler for Prompt Refinement	142
10.9	Summary	144
11	Improvement	145
11.1	Accuracy Optimization	145
11.2	Quick Wins and Best Practices for Cost Reduction	147
11.3	Leveraging LLM Provider Caching	149
11.4	Advanced Strategy: Implementing Model Cascades	150
11.5	Summary	153

How To Read This Book

This book is a companion to the course *AI Evals for Engineers and Product Managers* (<https://maven.com/parlance-labs/evals>). The course is designed for a broad audience ranging from engineers to technical product managers.

You will encounter code and math throughout the book. If you don't know how to code or are not familiar with the math, that's okay, just focus on the high-level concepts. You will gain tremendous value from understanding the process of evaluation, forming mental models, and recognizing common pitfalls.

We encourage you to engage with the examples and reflect on what makes evaluations work (or fail). Skip sections that feel out of scope from your role. When in doubt, ask questions in the course Discord. Think of this book as a flexible tool to support your learning.

Finally, this book is a rough draft that we will evolve over the next few months in response to student feedback. **We would appreciate it if you kept this book to yourself for now.** We will be working to publish an updated version of this book that is available to the public later this year.

1 Introduction

The past two years have seen rapid advances in the development and deployment of large language models (LLMs). Organizations are now embedding LLMs in critical pipelines in applications: customer service, content creation, decision support, and information extraction, among others (Bommasani et al. 2021; Zaharia et al. 2024). However, adoption is outpacing our ability to systematically evaluate LLM pipelines (Ward and Feldstein 2024).

Unlike traditional software, LLM pipelines do not produce deterministic outputs. Their responses are often subjective, context-dependent, and multifaceted. A response may be factually accurate but inappropriate (i.e., the “vibes are off”). They may sound persuasive while conveying incorrect information. These ambiguities make evaluation fundamentally different from conventional software testing and even traditional machine learning (ML).¹

The core challenge is as follows: How do we assess whether an LLM pipeline is performing adequately? And how do we diagnose where it is failing? Our course focuses on this gap. Although significant work has been done on model development², prompting techniques, and pipeline architectures, rigorous evaluation has received comparatively little attention. In this reader, we will build a practical and theoretical foundation for LLM evaluation, with methods that you can immediately apply to your own projects.³

¹ In traditional ML, outputs are well-defined—numbers, classes, or structured predictions. Standard metrics like accuracy or F1 score are widely used. Ground truth labels provide a clear target for evaluation. In contrast, LLM output can be open-ended, unstructured, and often subjective.

² If you are interested in foundation model evals, check out Liang et al. (2023), a long paper describing general metrics and evals, and some domain-specific evals for math, science, and coding: Hendrycks et al. (2021); Rein et al. (2024); Jain et al. (2025).

³ Our course is meant to be tool and model-agnostic; if a new LLM comes out tomorrow, our methodology will still be applicable.

Note to readers

This book focuses on evaluating LLM pipelines in specific applications. *We don't cover general-purpose foundation model training or benchmarking.* Our target audience is developers who integrate LLMs into real-world applications and need reliable evaluation methods. The approaches work with both commercial APIs and fine-tuned models.

1.1 What is Evaluation?

Before we dive into methods, we need a clear definition of evaluation.

Evaluation refers to systematic measurement of quality in an LLM pipeline. A good evaluation produces results that can be easily and unambiguously interpreted. Typically, this means a quantitative score, but it can also take the form of a structured qualitative summary or report.

Throughout this course, we will refer to an evaluation metric as an *eval*. A single pipeline will often use multiple evals to measure different aspects of performance. Evals can be operationalized in several ways. For example:

- *Background Monitoring* tracks evals passively over time to detect drift or degradation, without disrupting the primary workflow.
- *Guardrails* run evals in the critical path of the pipeline. If an eval fails, the pipeline can block the output, retry the operation, or fall back to a safer alternative before showing a result to the user.
- Evals can also be used to *improve pipelines*. For example, evals can label data for fine-tuning LLMs, select high-quality few-shot examples for prompts, or identify failure cases that motivate architectural changes.

Evals are essential not only to catch errors, but also to maintain user trust, ensure safety, monitor system behavior, and enable systematic improvement. Without them, teams are left guessing where failures occur and how best to fix them.

1.2 The Three Gulfs of LLM Pipeline Development

A useful framework for understanding LLM application development challenges is the “Three Gulfs” model (Figure 1), adapted from [Shankar et al. \(2025\)](#) and inspired by [Norman \(1988\)](#). This model captures the major gaps that developers must bridge when building any LLM pipeline.

Example 1 (Email Processing Pipeline). *Consider a pipeline that processes incoming emails sent to an organization. The goal is to extract the sender's name, summarize the key requests, and categorize the emails.*

At first glance, the task described in Example 1 seems simple. But each stage of development exposes new challenges across the Three Gulfs.

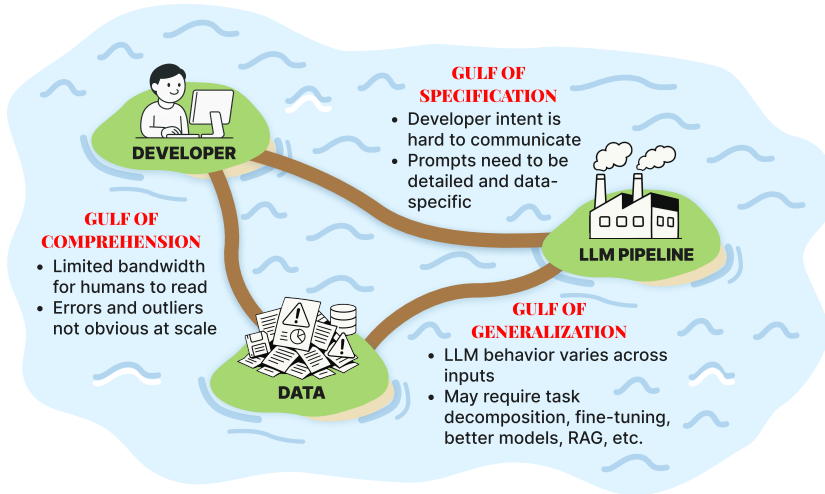


Figure 1: Three Gulfs Model of Challenges in LLM Pipeline Development. Adapted from *Steering Semantic Data Processing with DocWrangler* (Shankar et al. 2025).

The Gulf of Comprehension (Developer → Data) The first gulf separates us, the developer, from fully comprehending both our **data** and our **pipeline’s behavior** on that data. Understanding the data itself—the inputs the pipeline will encounter, such as real user queries or documents like the emails in Example 1—is the starting point. At scale, it is difficult to know the characteristics of this input data distribution, detect errors within it, or identify unusual patterns. Thousands of inputs might arrive daily, with diverse formats and varying levels of clarity. We cannot realistically review each one manually.⁴

Moreover, this gulf extends to understanding the LLM pipeline’s actual outputs and common failure modes when applied to this data. Just as we cannot read every input, we also cannot manually inspect every output trace generated by the pipeline to grasp all the subtle ways it might succeed or fail across the input space. So the dual challenge is: how can we understand the important properties of our input data, *and* the spectrum of our pipeline’s behaviors and failures on that data, *without examining every single example*?

The Gulf of Specification (Developer → LLM Pipeline) The second gulf separates what we mean from what we actually specify in the pipeline. Our intent—the task we want the LLM to perform—is often only *loosely* captured by the prompts you write. Specifying tasks precisely in natural language is hard. Prompts must be detailed enough to remove ambiguity, and aligned with the structure and diversity of the data.

Even prompts that seem clear often leave crucial details unstated. For example, we might write:

Extract the sender’s name and summarize the key requests in this email.

⁴ **Hamel’s Note:** The gulf of comprehension represents the fact that you must know what you want so you can effectively prompt a LLM, evaluate it, and debug it. Additionally, this process is iterative; our research and experience show that people refine their requirements to adapt to the behavior of the LLM. Shankar et al. (2024d). We discuss how to systematically look at data to inform your prompts in future sections.

At first glance, this sounds specific. But important questions are left unanswered:

- Should the summary be a paragraph or a bulleted list?
- Should the sender be the display name, the full email address, or both?
- Should the summary include implicit requests, or only explicit ones?
- How concise or detailed should the summary be?

The LLM cannot infer these decisions unless we explicitly specify them.⁵ Without complete instructions, the model is forced to “guess” our true intent, often resulting in inconsistent outputs. The Gulf of Specification captures this gap between what we want and what we actually communicate.⁶

The Gulf of Generalization (Data → LLM Pipeline) The third gulf separates our data from the pipeline’s generalization behavior. Even if prompts are carefully written, LLMs may behave inconsistently across different inputs.

In the email processing example, imagine an email that mentions a public figure, like *Elon Musk* or *Donald Trump*, within the body text. The model might mistakenly extract these names as the sender, even though they are unrelated to the actual email metadata.⁷ This is not a prompting error. It is a generalization failure: the model applies the instructions incorrectly because it has not generalized properly across diverse data.

The Gulf of Generalization reminds us that even when prompts are clear and well-scoped, LLMs may still exhibit biases, inconsistencies, or unexpected behaviors when encountering new or unusual inputs. Even as models get better, the Gulf of Generalization will always exist to some degree, because no model will ever be perfectly accurate on all inputs (Kalai and Vempala 2024).

1.3 Why LLM Pipeline Evaluation is Challenging

Developing effective evaluations for LLM pipelines is hard. It is a sufficiently different problem from traditional software engineering and machine learning operations (MLOps) that new approaches are required.

First, each application requires bridging the Three Gulfs anew. The specific ways in which understanding fails, prompts fall short, or generalization breaks down are unique to our task and dataset. There are no universal evaluation recipes.

Second, requirements often emerge only after interacting with early outputs. We might initially expect summaries to be written in prose but later realize that bulleted lists are easier for users to scan. Evaluation criteria must evolve alongside system development (Shankar et al. 2024d).

⁵ **Hamel’s Note:** Underspecified prompts are usually a direct result of not looking at the data (ignoring the “data” island).

⁶ **Shreya’s Note:** In LLM development, prompt clarity surprisingly often matters as much as task complexity.

⁷ The pipeline described in <https://www.docetl.org/showcase/ai-rfi-response-analysis> exhibits this failure mode.

Third, appropriate metrics are rarely obvious at the outset. Unlike traditional software, where correctness is well-defined, LLM pipelines involve tradeoffs: factual accuracy, completeness, conciseness, style, and more. Choosing the right metrics depends on the specific goals of our application—and often requires experimentation.

Finally, there is no substitute for examining real outputs on real data. Generic benchmarks cannot capture the specific failure modes of our pipeline. For Example 1, without inspecting a diverse set of extracted senders, we would not notice that the model sometimes misidentifies public figures as senders. Systematic evaluation requires careful, hands-on analysis of representative examples.⁸

1.4 The LLM Evaluation Lifecycle: Bridging the Gulfs with Evaluation

⁸ 📌 **Hamel's Note:** If you are not willing to look at some data manually on a regular cadence you are wasting your time with evals. Furthermore, you are wasting your time more generally.

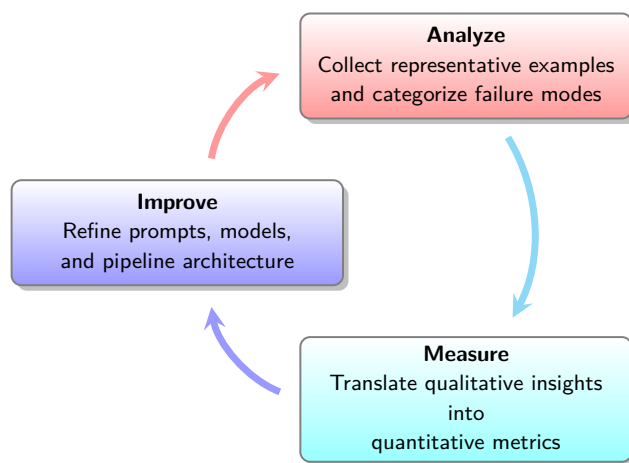


Figure 2: The Analyze-Measure-Improve evaluation lifecycle.

The Three Gulfs—Comprehension, Specification, and Generalization—highlight **the core challenges** inherent in developing reliable LLM pipelines. **Evaluation provides the systematic means to understand and address these challenges.** In this course, we introduce a practical, iterative approach centered around evaluation: the **Analyze-Measure-Improve** lifecycle, depicted in Figure 2. This lifecycle provides a repeatable method for using evaluation to build, validate, and refine LLM pipelines, specifically aimed at overcoming the Three Gulfs.

The lifecycle begins with **Analyze**, directly tackling the *Gulf of Comprehension*. We inspect the pipeline’s behavior on representative data to *qualitatively* identify failure modes. This critical first step illuminates why the pipeline might be struggling. Failures uncovered often point clearly to ambiguous instructions (Specification issues) or inconsistent performance across inputs (Generalization issues).

The findings from Analyze guide our next actions. Failures stemming directly from poor Specification—unclear prompts or ambiguous

instructions—can often be addressed immediately in the **Improve** phase. We simply make the instructions clearer. However, other failures, particularly those suggesting inconsistent generalization or complex interactions, require deeper investigation. Understanding their true frequency, impact, and root causes demands quantitative data before effective improvements can be made.

This need for data drives the **Measure** phase. Here, we develop and deploy specific evaluators (evals) to *quantitatively* assess the failure modes identified in Analyze, especially those needing further scrutiny. Measurement provides hard numbers on failure rates and patterns. This data is crucial for prioritizing which problems to fix first and for diagnosing the underlying causes of tricky generalization failures.

Finally, the **Improve** phase leverages the findings from both previous phases to actively bridge the *Gulfs of Specification and Generalization*. We make targeted interventions. This includes direct fixes to prompts and instructions addressing Specification issues identified during Analyze. It also involves data-driven efforts—guided by the quantitative results from Measure—such as engineering better examples, refining retrieval strategies, adjusting architectures, or fine-tuning models to enhance generalization.

Cycling through Analyze, Measure, and Improve (Figure 2) creates a powerful feedback loop. It uses structured evaluation to systematically navigate the complexities posed by the Three Gulfs, leading to more reliable and effective LLM applications. We will cover each phase of the evaluation lifecycle in greater depth in the upcoming sections.

1.5 Summary

Effective evaluation is the foundation of reliable LLM applications. Without it, development reduces to guesswork. The key takeaways from this lecture are:

- Evaluation is not optional. It is essential for any serious LLM application.
- The Three Gulfs model helps categorize and diagnose different sources of failure.
- Choosing appropriate metrics requires examining our specific data and use case—not relying on generic benchmarks.
- The Analyze-Measure-Improve lifecycle provides a structured, repeatable method for building better evaluations over time.

In the next sections, we will dive deeper into evaluation metrics and systematic error analysis techniques.

About This Preview

The table of contents and this first chapter are a preview of what we teach in our course. **Students enrolled in our course get access to the full version.** The course reader is designed to give students comprehensive notes that supplements their learning. We suggest revisiting the table of contents to get a sense of the breadth of material covered. **To see student reactions to these materials (and secret discount codes), see this page <https://bit.ly/eval-reviews>.**